

Numerische Methoden

Komplettes Lernskript – allgemein, mit Graphen, für jede Klausur

Erklärt *was* es ist, *wie* es geht und *wann* man es nutzt – pro Thema ein durchgerechnetes Beispiel.

Dein Stand pro Thema (aus der letzten Probeklausur): **stark** = im Test gut **teils** = kleine Lücke **üben!**
= war Lücke **ungetestet** = noch nicht geprüft

Inhaltsverzeichnis

1	Zahlendarstellung & Fehler	1
1.1	Warum überhaupt Fehler?	2
1.2	Gleitkomma, Maschinenepsilon, Auslöschung	2
2	Fehleranalyse: die vier Eigenschaften	2
2.1	Die Begriffe (wer wird mit wem verglichen?)	2
3	Newton- & Sekantenverfahren	3
3.1	Was & warum	3
3.2	Herleitung aus Taylor (musst du können)	3
3.3	Konvergenz, Kriterium, Versagen	3
4	Optimierung	4
4.1	Was & warum	4
4.2	Verfahren	4
5	Differenzialgleichungen (Euler & Runge-Kutta)	4
5.1	Was & warum	4
5.2	Runge-Kutta & Ordnung	5
6	Integration & Interpolation	5
6.1	Numerische Integration (Newton-Cotes)	5
6.2	Interpolation	6
7	Fourier-Analyse & FFT	6
7.1	Was & warum	6
8	Modelltraining (Numerik trifft ML)	6
8.1	Was & warum	7
8.2	Zwei Dinge, die immer drankommen	7

1 Zahlendarstellung & Fehler

stark

1.1 Warum überhaupt Fehler?

Ein Rechner hat *endlich* viele Bits. Reelle Zahlen müssen also *gerundet* werden, und viele Verfahren brechen einen Grenzprozess ab. Jedes numerische Ergebnis trägt deshalb einen Fehler. Man unterscheidet drei Quellen:

- **Modellfehler:** die Realität wird vereinfacht (z. B. Reibung ignoriert).
- **Abschneide-/Diskretisierungsfehler:** ein Grenzwert wird abgebrochen (Reihe, Ableitung $\rightarrow$ Differenzenquotient). Skaliert mit der Schrittweite h .
- **Rundungsfehler:** endliche Stellenzahl der Maschine.

Absoluter Fehler $|x - \tilde{x}|$, **relativer Fehler** $\frac{|x - \tilde{x}|}{|x|}$ (skaleninvariant – „wie viele gültige Stellen“; bei $x \approx 0$ nur absolut sinnvoll).

1.2 Gleitkomma, Maschinenepsilon, Auslöschung

Eine Gleitkommazahl ist $x = (-1)^s m \cdot 2^e$: Vorzeichen s , *Mantisse* m (signifikante Stellen $\rightarrow$ **Auflösung**), *Exponent* e (Skalierung $\rightarrow$ **Wertebereich**). Normalisiert heißt $m = 1.b_1b_2\dots$ mit implizitem führenden Bit. Die zentrale Kennzahl ist das **Maschinenepsilon** $\varepsilon = 2^{-t}$ (t = Mantissen-Nachkommabits): der Abstand von 1 zur nächsten darstellbaren Zahl.

Wichtig ist das Bild der *Abstände*: darstellbare Zahlen liegen nicht gleichmäßig, sondern werden mit jeder Zweierpotenz *doppelt so weit* auseinander – konstante *relative*, wachsende *absolute* Auflösung:



Gleich viele Zahlen pro „Oktave“ $[1, 2), [2, 4), [4, 8)$ – der absolute Abstand verdoppelt sich.

Auslöschung (catastrophic cancellation): subtrahiert man zwei fast gleiche Zahlen, löschen sich die führenden Stellen weg, übrig bleibt Rauschen. Gegenmittel: *umformen*, sodass addiert statt subtrahiert wird, z. B. $\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$. Festkomma dagegen hat *konstante absolute* Auflösung 2^{-f} (gleichmäßiges Raster), dafür kleinen Wertebereich.

Durchgerechnet – Maschinenepsilon & warum eine Differenz „explodiert“

System mit $t = 2$ Mantissenbits $\Rightarrow \varepsilon = 2^{-2} = 0,25$. Darstellbar ab 1: 1,00; 1,25; 1,50; 1,75. Was ist $(1+0,1) - 1$? Da 0,1 kleiner als die halbe Auflösung 0,125 ist, rundet $1,1 \rightarrow 1,0$. Also $(1+0,1) - 1 = 1,0 - 1,0 = 0$ – der ganze Beitrag 0,1 ist weg. Eine anschließende Division $1/0$ liefert ∞ . Lehre: kleine Zahlen *zuerst* zusammenfassen, kritische Differenzen umformen.

2 Fehleranalyse: die vier Eigenschaften

teils

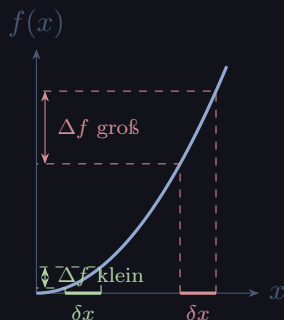
2.1 Die Begriffe (wer wird mit wem verglichen?)

Notation: f = wahres Problem, $\hat{f}$ = Algorithmus, $\tilde{x}$ = gestörte Eingabe.

- **Kondition** $\kappa = |f(x) - f(\tilde{x})|$: Empfindlichkeit des *Problems* gegenüber Eingabestörung. *Gehört zum Problem*. Faustformel $\kappa \approx |f'(x)| \cdot \delta x$.
- **Stabilität** $|\hat{f}(\tilde{x}) - \hat{f}(x)| \leq s |\tilde{x} - x|$: verstärkt der *Algorithmus* Störungen? *Gehört zum Verfahren*.
- **Konsistenz** $c = |\hat{f}(x) - f(x)|$: wie gut nähert die Diskretisierung bei *exakter* Eingabe? Skaliert typischerweise mit h .
- **Konvergenz** $|\hat{f}(\tilde{x}) - f(x)|$: der *Gesamtfehler*.

Merksatz: Konsistenz + Stabilität $\Rightarrow$ Konvergenz.

Die Kondition versteht man am besten geometrisch: an einer *flachen* Stelle erzeugt eine Eingabestörung δx nur eine kleine Ausgangsänderung (gut konditioniert), an einer *steilen* Stelle eine große (schlecht konditioniert):



Gleiches δx : flach $\rightarrow$ kleines Δf (gut konditioniert), steil $\rightarrow$ großes Δf (schlecht).

Durchgerechnet – Konsistenz auf zwei Gittern

$g(x) = (x - 2)^2$, am nächsten Gitterpunkt ausgewertet. An $x = 2,7$ (echt $g = 0,49$): grobes Gitter $h = 1$ rundet auf 3, $\hat{g} = g(3) = 1 \Rightarrow c = 0,51$; feines Gitter $h = \frac{1}{2}$ rundet auf 2,5, $\hat{g} = g(2,5) = 0,25 \Rightarrow c = 0,24$. Kleineres h ist konsistenter. **Aber:** unter einem optimalen h dominieren Rundungsfehler – kleiner ist *nicht* automatisch besser.

3 Newton- & Sekantenverfahren

teils

3.1 Was & warum

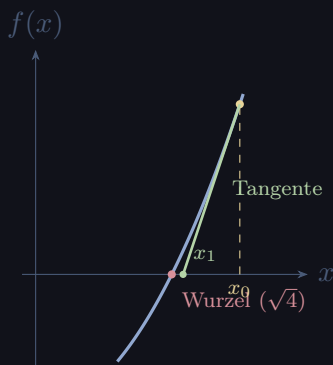
Gesucht ist eine **Nullstelle** $f(x) = 0$, wenn keine Formel existiert. Newton nutzt die *Tangente*: man ersetzt f lokal durch ihre Linearisierung und springt zu deren Nullstelle.

3.2 Herleitung aus Taylor (musst du können)

Taylor 1. Ordnung um x_n : $f(x) \approx f(x_n) + f'(x_n)(x - x_n)$. Diese Tangente = 0 setzen und nach x auflösen:

$$0 = f(x_n) + f'(x_n)(x - x_n) \implies x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Geometrisch: lege die Tangente an, lies ab, wo sie die x -Achse schneidet, wiederhole:



Tangente in $x_0 \rightarrow$ ihre Nullstelle = x_1 (näher an der Wurzel). Wiederholen.

3.3 Konvergenz, Kriterium, Versagen

Quadratische Konvergenz $|e_{n+1}| \leq C|e_n|^2$: die Zahl korrekter Stellen *verdoppelt* sich pro Schritt. **Kriterium:** x_0 nah genug an einer *einfachen* Wurzel, $f \in C^2$, $f' \neq 0$ dort. **Versagen:** $f'(x_n) = 0$

(Division durch 0), schlechter Start (Divergenz), oder ein Zyklus. **Sekantenverfahren**: ersetzt f' durch die Vorwärtsdifferenz $\frac{f(x_0+h)-f(x_0)}{h}$ (Fehler $\mathcal{O}(h)$) bzw. durch die Steigung der letzten zwei Punkte – ableitungsfrei, dafür langsamer (Ordnung $\approx 1,618$) und zwei Startwerte nötig.

Durchgerechnet – Newton für $\sqrt{2}$ ($f = x^2 - 2$, $f' = 2x$, $x_0 = 2$)

$$x_1 = 2 - \frac{2^2 - 2}{2 \cdot 2} = 2 - \frac{1}{2} = \frac{3}{2}, \quad x_2 = \frac{3}{2} - \frac{(\frac{3}{2})^2 - 2}{2 \cdot \frac{3}{2}} = \frac{3}{2} - \frac{1/4}{3} = \frac{17}{12} \approx 1,4167.$$

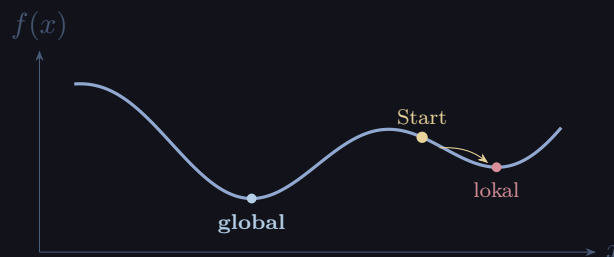
Wichtig: $f'(\frac{3}{2}) = 2 \cdot \frac{3}{2} = 3$ im Nenner. ($\sqrt{2} \approx 1,4142$ – nach 2 Schritten 3 Stellen.)

4 Optimierung

stark

4.1 Was & warum

Gesucht ist ein **Minimum** von f . Ein *lokales* Minimum ist der kleinste Wert in einer Umgebung, das *globale* der kleinste überhaupt. Lokale Verfahren (Gradientenabstieg, Newton) folgen nur lokaler Steigung und laufen ins *nächste* Tal – bei vielen Tälern verfehlen sie das globale:



Ball rollt ins *nächste* (lokale) Tal $x \approx 6,6$ – verfehlt das tiefere globale bei $x \approx 3,1$.

4.2 Verfahren

Gradientenabstieg $x_{n+1} = x_n - \eta f'(x_n)$ (Schritt bergab; η zu groß $\rightarrow$ Oszillation, zu klein $\rightarrow$ langsam). **Newton fürs Optimum** $x_{n+1} = x_n - \frac{f'(x_n)}{|f''(x_n)|}$ – der Betrag $|f''|$ erzwingt einen Schritt *immer bergab* (sonst liefe man bei negativer Krümmung zu einem Maximum); Klassifikation per f'' : > 0 Minimum, < 0 Maximum. **Global**: Random Restarts ($P = 1 - (1 - p)^K$), Simulated Annealing (akzeptiert Verschlechterung mit $e^{-\Delta E/T}$, Abkühlen $T \downarrow$), Basin Hopping. Bei hochfrequenten Störtermen hilft *Glätten*: Schrittweite $h \approx$ Periode des Störterms wählen, dann mittelt sich die Oszillation heraus.

Durchgerechnet – Konvexität & Random Restarts

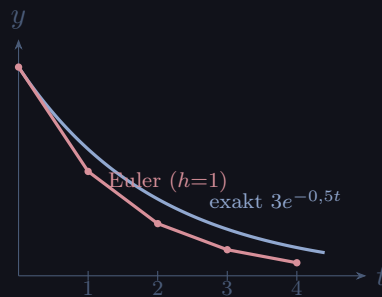
Ist f **konvex** ($f'' \geq 0$ überall), gibt es nur *ein* Minimum $\Rightarrow$ jedes lokale ist global, lokale Verfahren genügen. Sonst: K zufällige Starts, Trefferchance $p = \frac{1}{2}$ pro Start. $P(\geq 1 \text{ Treffer}) = 1 - (1 - \frac{1}{2})^K$. Für $P > 0,9$: $(\frac{1}{2})^K < 0,1 \Rightarrow 2^K > 10 \Rightarrow K = 4$ ($P = \frac{15}{16}$).

5 Differenzialgleichungen (Euler & Runge-Kutta)

ungetestet

5.1 Was & warum

Ein Anfangswertproblem $y' = f(t, y)$, $y(t_0) = y_0$ hat oft keine geschlossene Lösung. Man läuft in Schritten der Weite h voran und schätzt die nächste Stelle aus der Steigung f . **Explizites Euler**: $y_{n+1} = y_n + h f(t_n, y_n)$ – billig, aber nur für kleine h stabil (für $y' = -\lambda y$: stabil nur bei $0 < h < \frac{2}{\lambda}$). **Implizit** nutzt $f(t_{n+1}, y_{n+1})$ – teurer (Gleichung lösen), dafür stabil auch bei großem h (wichtig bei *steifen* Problemen).



Euler (rose) folgt geraden Tangentenstücken und liegt unter der Kurve – RK4 wertet f viermal pro Schritt aus und trifft fast exakt.

5.2 Runge-Kutta & Ordnung

RK4 kombiniert vier Steigungen: $y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ mit $k_1 = f(t_n, y_n)$, $k_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1)$, $k_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2)$, $k_4 = f(t_n + h, y_n + hk_3)$. **Konvergenzordnung** p : globaler Fehler $\mathcal{O}(h^p)$ – Euler $p=1$, RK4 $p=4$ (darum bei gleichem h enorm genauer). Ein **Butcher-Tableau** $\begin{array}{c|c} c & A \\ \hline b \end{array}$ kodiert Knoten c_i , Gewichte b_i , Kopplung a_{ij} ; *explizit* $\Leftrightarrow A$ streng untere Dreiecksmatrix.

Durchgerechnet – Ein Euler- vs. ein RK4-Schritt ($y' = -y$, $y_0 = 1$, $h = 1$, **exakt** $e^{-1} = 0,368$)

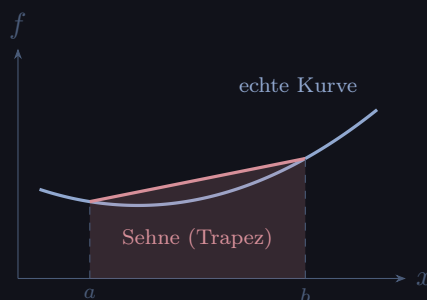
Euler: $y_1 = 1 + 1 \cdot (-1) = 0$. RK4: $k_1 = -1$, $k_2 = f(1 - \frac{1}{2}) = -0,5$, $k_3 = f(1 - 0,25) = -0,75$, $k_4 = f(1 - 0,75) = -0,25$, $y_1 = 1 + \frac{1}{6}(-1 - 1 - 1,5 - 0,25) = 0,375$. RK4 trifft fast exakt, Euler weit daneben.

6 Integration & Interpolation

üben!

6.1 Numerische Integration (Newton-Cotes)

Idee: $\int_a^b f dx$ approximieren, indem man f durch ein einfaches Polynom über Stützpunkte ersetzt und *das* integriert. Gerade $\rightarrow$ **Trapez**, Parabel $\rightarrow$ **Simpson**. Der *Exaktheitsgrad* ist der höchste Polynomgrad, der noch fehlerfrei integriert wird (Trapez 1, Simpson 3). Geometrisch ersetzt Trapez die Kurve durch eine Sehne – der Spalt dazwischen ist der Fehler; Simpson legt eine Parabel an und trifft Krümmung viel besser:



Der Spalt zwischen Sehne und Kurve ist der Trapez-Fehler – bei Krümmung groß.

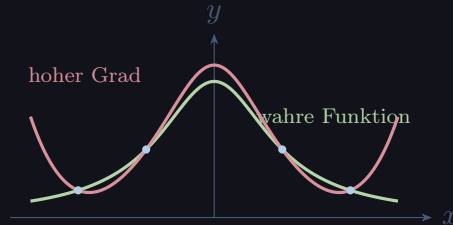
Zusammengesetzte Gewichte (für $[0, 2]$ mit f_0, f_1, f_2 , $h = 1$):

$$\text{Trapez: } \frac{h}{2} [f_0 + 2f_1 + f_2], \quad \text{Simpson: } \frac{h}{3} [f_0 + 4f_1 + f_2] \text{ („1,4,1 durch 3“)}.$$

Genau ein innerer Term, mit Gewicht 2 bzw. 4 – nicht $2(f_1 + f_2)$, keine Extra-Terme. Weiter: **3/8-Regel** (Gewichte 1, 3, 3, 1), **Boole** (7, 32, 12, 32, 7), **Romberg** (Trapez für $h, \frac{h}{2}$ extrapolieren: $R = \frac{4T(h/2) - T(h)}{3}$), **Monte-Carlo** ($\hat{I} = |\Omega| \frac{1}{N} \sum f(X_i)$, Fehler $\propto 1/\sqrt{N}$, dimensionsunabhängig $\Rightarrow$ schlägt das Gitter (n^d Punkte) in hohen Dimensionen).

6.2 Interpolation

Durch $n+1$ Punkte gibt es *genau ein* Polynom vom Grad $\leq n$. **Lagrange**: $p(x) = \sum_i y_i L_i(x)$ mit $L_i(x) = \prod_{j \neq i} \frac{x-x_j}{x_i-x_j}$ (L_i ist 1 am eigenen Knoten, sonst 0). **Vorsicht hoher Grad**: auf gleichmäßigen Knoten oszilliert das Polynom an den Rändern stark – das **Runge-Phänomen**. Darum nicht den Grad hochdrehen, sondern *zusammengesetzte* Regeln oder **Splines** (stückweise niedriger Grad, glatt):



Runge: das Polynom (rose) trifft die Knoten (•), schwingt aber an den Rändern stark über – Splines vermeiden das.

Durchgerechnet – Lagrange durch $(0, 1), (1, 2), (2, 5)$

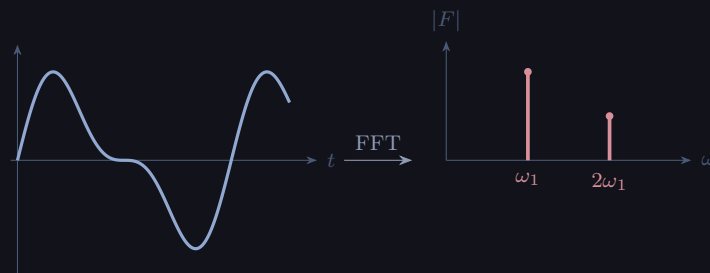
$L_0 = \frac{(x-1)(x-2)}{(0-1)(0-2)} = \frac{(x-1)(x-2)}{2}$, $L_1 = \frac{x(x-2)}{(1)(-1)} = -x(x-2)$, $L_2 = \frac{x(x-1)}{(2)(1)} = \frac{x(x-1)}{2}$. Dann $p = 1 \cdot L_0 + 2 \cdot L_1 + 5 \cdot L_2 = x^2 + 1$ (Probe: $1, 2, 5$ ✓), $p(\frac{1}{2}) = \frac{5}{4}$.

7 Fourier-Analyse & FFT

ungetestet

7.1 Was & warum

Die Fourier-Transformation $F(\omega) = \int f(t)e^{-i\omega t} dt$ zerlegt ein Zeitsignal in seine **Frequenzen**: $|F(\omega)|$ sagt, wie stark Frequenz ω enthalten ist; die Rücktransformation baut das Signal wieder zusammen. Mit der Euler-Formel $\cos(\omega t) = \frac{1}{2}(e^{i\omega t} + e^{-i\omega t})$ liefert jeder reelle Kosinus *zwei* Linien bei $\pm\omega$. Ein Summensignal („Interferenz“) zeigt also pro Schwingung einen Peak:



Summensignal (links) → Spektrum (rechts): zwei Peaks bei $\omega_1, 2\omega_1$, Höhen $1 : \frac{1}{2}$ wie die Amplituden.

DFT: $X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}$ (N diskrete Bins), Kosten $\mathcal{O}(N^2)$. Die **FFT** spaltet rekursiv in gerade/ungerade Indizes $\Rightarrow \mathcal{O}(N \log N)$. Bin k entspricht Frequenz $f_k = k \frac{f_s}{N}$; die **Nyquist**-Frequenz $f_s/2$ ist die höchste eindeutig darstellbare – höhere erscheinen als *Aliasing* verfälscht.

Durchgerechnet – DFT von $x = (1, 0, 1, 0)$, $N = 4$

$X_0 = 1+0+1+0 = 2$; $X_1 = 1 + 1 \cdot e^{-i\pi} = 1 - 1 = 0$; $X_2 = 1 + 1 \cdot e^{-i2\pi} = 2$; $X_3 = 0$. Also $|X| = (2, 0, 2, 0)$: Gleichanteil (X_0) plus schnellste Schwingung (X_2 , Nyquist) – genau die Struktur von $(1, 0, 1, 0)$.

8 Modelltraining (Numerik trifft ML)

üben!

8.1 Was & warum

Ein lineares Modell $\hat{y}(x) = wx + b$ wird trainiert, indem ein **Verlust** minimiert wird – also Optimierung (Abschnitt 4) auf einer Verlustfunktion. Standard ist der **mittlere quadratische Fehler**

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (wx_i + b - y_i)^2.$$

Die Abweichungen werden **quadriert** – sonst heben sich $+/-$ auf; das Quadrat bestraft große Fehler stärker und macht L glatt/konvex. Trainiert wird per Gradientenabstieg.

8.2 Zwei Dinge, die immer drankommen

(1) **Erster Loss = Varianz.** Mit $w = 0$, $b = \bar{y}$ sagt das Modell überall den Mittelwert voraus, also $L = \frac{1}{n} \sum (\bar{y} - y_i)^2 = \text{Var}(y)$ – die Varianz ist das Mittel der *quadrierten* Abweichungen. (2) **Numerik-Brille auf ML-Praktiken:**

- Überparametrisiertes Modell soll eine kleine Teilmenge auf Loss 0 bringen – klappt das nicht, ist es ein *Bug* (Modell/Loss/Daten), nicht der Optimierer (wie „Solver an bekannter Lösung testen“).
- Mini-Batch-SGD $\leftrightarrow$ Monte-Carlo (Gradient als Stichprobenmittel, Fehler $\propto 1/\sqrt{B}$).
- Mehrere Seeds, bester Lauf $\leftrightarrow$ Random Restarts.
- Quantisierung $\leftrightarrow$ Präzisionswahl im Gleitkomma (kleinstes ausreichendes Format).

Durchgerechnet – Varianz nicht ohne Quadrat!

$y = (2, 4, 6)$, $\bar{y} = 4$. Falsch: $\frac{1}{3}(-2 + 0 + 2) = 0$. Richtig: $\text{Var} = \frac{1}{3}[(-2)^2 + 0^2 + 2^2] = \frac{1}{3}(4 + 0 + 4) = \frac{8}{3} \approx 2,67$. Das ist der erwartete erste Loss; ein gedruckter Wert ≈ 100 verrät einen Bug/falsche Initialisierung.

Stand-Legende: **stark** Themen 1,4 · **teils** Themen 2,3 · **üben!** Themen 6,8 · **ungetestet** Themen 5,7. Letztere zwei kamen in der letzten Klausur nicht dran, gehören aber zum Stoff – für die nächste Klausur mitnehmen.